

Lab 6 Solution

Class:

```
class stimulus;

// -----
// class members (Req 1)
// -----
rand logic [7:0] A;
rand logic [7:0] B;
rand logic [3:0] ALU_FUN;
rand logic      enable;
rand logic      RST;
bit            clk;          // non-random, mirrors tb clock

// walking-ones position
bit [2:0] idx;

// -----
// Req 2: RST asserted (RST == 0, since 1 = "reset released") with
// low probability
// -----
constraint c_rst {
    RST dist { 1'b0 := 5,    // reset asserted -- rare
              1'b1 := 95 }; // reset released -- common
}

// -----
// Req 3: enable -- valid cases (1) far more frequent than invalid (0)
// -----
constraint c_enable {
    enable dist { 1'b0 := 20,
                 1'b1 := 80 };
}

// -----
// Req 4: ALU_FUN -- all valid opcodes can appear, invalid (0xF) rare
// -----
constraint c_alufun {
    ALU_FUN dist { [4'h0 : 4'hE] := 10,
                   4'hF         := 1 };
}

// -----
// Req 5: ADD / MUL -- boundary values (0x00, 0x7F, 0x80) hit more
// often than the rest of the range
// -----
constraint c_ab_add {
    (ALU_FUN == 4'h0) -> {
        A dist { 8'h7F := 5,
                 8'h00 := 5,
                 8'h80 := 5,
                 [8'h01:8'h7E] := 1,
                 [8'h81:8'hFF] := 1 };

        B dist { 8'h7F := 5,
                 8'h00 := 5,
                 8'h80 := 5,
                 [8'h01:8'h7E] := 1,
                 [8'h81:8'hFF] := 1 };
    }
}
```

```

constraint c_ab_mul {
  (ALU_FUN == 4'h2) -> {
    A dist { 8'h7F      := 5,
             8'h00      := 5,
             8'h80      := 5,
             [8'h01:8'h7E] := 1,
             [8'h81:8'hFF] := 1 };

    B dist { 8'h7F      := 5,
             8'h00      := 5,
             8'h80      := 5,
             [8'h01:8'h7E] := 1,
             [8'h81:8'hFF] := 1 };
  }
}

// -----
// Reg 6: OR / XOR -- A walks a single active bit (0000_0001,
// 0000_0010, ... 1000_0000, wrapping), driven by idx below;
// B is mostly zero.
// -----
constraint c_ab_or {
  (ALU_FUN == 4'h5) -> {
    A = (8'b1 << idx);
    B dist { 8'h00      := 8,
             [8'h01:8'hFF] := 1 };
  }
}

constraint c_ab_xor {
  (ALU_FUN == 4'h8) -> {
    A = (8'b1 << idx);
    B dist { 8'h00      := 8,
             [8'h01:8'hFF] := 1 };
  }
}

// Reg 7: SHR (0xD) / SHL (0xE) -- no constraint block for them at
// all, so A and B are left fully free.

// -----
// Reg 8: pre_randomize -- show values before the solver runs
// -----
function void pre_randomize();
  $display("\n //----- BEFORE RANDOMIZATION -----");
  $display("      RST = %0b", RST);
  $display("    enable = %0b", enable);
  $display("    ALU_FUN = 4'h%01h (%s)", ALU_FUN, fun_name(ALU_FUN));
  $display("      A = 8'h%02h", A);
  $display("      B = 8'h%02h", B);
endfunction

```

```

// -----
// Req 8: post_randomize -- advance the walking-ones index for the
// next call, then show values after the solver finishes
// -----
function void post_randomize();
    idx = (idx + 1) % 8; // advance walking-one position for next call

    $display("\n ////////// AFTER RANDOMIZATION //////////");
    $display("      RST = %0b", RST);
    $display("      enable = %0b", enable);
    $display("      ALU_FUN = 4'h%01h (%s)", ALU_FUN, fun_name(ALU_FUN));
    $display("      A = 8'h%02h", A);
    $display("      B = 8'h%02h", B);
endfunction

// -----
// Req 9: drive -- apply inputs at negedge, wait for output to settle
// -----
task drive(ref logic [7:0] dut_A,
           ref logic [7:0] dut_B,
           ref logic [3:0] dut_FUN,
           ref logic      dut_en,
           ref logic      dut_RST);
    @(negedge clk);
    dut_A = A;
    dut_B = B;
    dut_FUN = ALU_FUN;
    dut_en = enable;
    dut_RST = RST;
    $display(" [drive] RST=%0b en=%0b FUN=%s A=8'h%02h B=8'h%02h",
             RST, enable, fun_name(ALU_FUN), A, B);
    @(negedge clk); // wait one clock for output to settle
endtask

// -----
// helper: opcode mnemonic string (used for display / self-check)
// -----
function string fun_name(logic [3:0] f);
    case (f)
        4'h0: return "ADD ";
        4'h1: return "SUB ";
        4'h2: return "MUL ";
        4'h3: return "DIV ";
        4'h4: return "AND ";
        4'h5: return "OR  ";
        4'h6: return "NAND";
        4'h7: return "NOR ";
        4'h8: return "XOR  ";
        4'h9: return "XNOR";
        4'hA: return "EQ  ";
        4'hB: return "GT  ";
        4'hC: return "LT  ";
        4'hD: return "SHR ";
        4'hE: return "SHL ";
        default: return "INV ";
    endcase
endfunction
endclass

```

Testbench:

```
`include "stimulus_class.sv"

module top;

    // -----
    // signals
    // -----
    bit        clk;
    logic       RST;
    logic [7:0] A, B;
    logic [3:0] ALU_FUN;
    logic       enable;
    logic       ALU_OUT_VALID;
    logic [15:0] ALU_OUT;

    // stimulus object (Req 11: instantiate the stimulus class)
    stimulus stim;

    // -----
    // DUT instantiation
    // -----
    ALU dut (
        .CLK        (clk),
        .RST        (RST),
        .A          (A),
        .B          (B),
        .ALU_FUN     (ALU_FUN),
        .enable      (enable),
        .ALU_OUT_VALID (ALU_OUT_VALID),
        .ALU_OUT     (ALU_OUT)
    );

    // -----
    // Req 11: clock generation
    // -----
    initial begin
        clk = 0;
        forever begin
            #5ns clk = ~clk;
            stim.clk = clk;
        end
    end
end
```

```

// -----
function automatic logic [15:0] golden_model(
    input logic [7:0] a,
    input logic [7:0] b,
    input logic [3:0] fun,
    input logic      en
);
    if (!en)
        return 16'h0000;
    case (fun)
        4'h0: return a + b;
        4'h1: return a - b;
        4'h2: return a * b;
        4'h3: return (b == 'h00) ? 16'hFFFF : (a / b);
        4'h4: return a & b;
        4'h5: return a | b;
        4'h6: return ~(a & b);
        4'h7: return ~(a | b);
        4'h8: return a ^ b;
        4'h9: return ~(a ^ b);
        4'hA: return (a == b) ? 16'h000A : 16'h0000;
        4'hB: return (a > b) ? 16'h000B : 16'h0000;
        4'hC: return (a < b) ? 16'h000C : 16'h0000;
        4'hD: return a >> 1;
        4'hE: return a << 1;
        default: return 16'h0000;
    endcase
endfunction

// -----
// self-check task: compares DUT output against golden_model()
// -----
task automatic check(
    input logic [7:0] a,
    input logic [7:0] b,
    input logic [3:0] fun,
    input logic      en,
    input logic      rst
);
    logic [15:0] exp;
    if (!rst || !en) return; // no output expected

    exp = golden_model(a, b, fun, en);
    if (ALU_OUT == exp)
        $display(" [PASS] %s A=%02h B=%02h OUT=%04h",
            stim.fun_name(fun), a, b, ALU_OUT);
    else
        $display(" [FAIL] %s A=%02h B=%02h OUT=%04h EXP=%04h",
            stim.fun_name(fun), a, b, ALU_OUT, exp);
endtask

```

```

// -----
// Req 11: main test sequence -- reset, then randomize/drive/check
// -----
initial begin

    stim = new();

    // apply reset
    RST = 0;
    enable = 0;
    A = 0;
    B = 0;
    ALU_FUN = 0;
    $display("\n[TB] reset asserted\n");
    repeat (3) @(posedge clk);
    RST = 1;
    @(posedge clk);
    $display("[TB] reset released\n");

    // randomize, drive and self-check
    repeat (50) begin
        assert(stim.randomize()) else $fatal(1, "randomize() failed");
        stim.drive(A, B, ALU_FUN, enable, RST);
        check(A, B, ALU_FUN, enable, RST);
    end

    $display("\n[TB] done\n");
    $stop;
end

endmodule

```

Do file:

```

if {[file exists work]} { vdel -lib work -all }
vlib work

vlog ALU.v
vlog stimulus_class.sv top.sv

vsim -voptargs=+acc work.top -sv_seed random

add wave *

run -all

```